Prompt: app Android de tradução de legendas embutidas em vídeo

Copie tudo abaixo da linha e cole na IA que vai gerar o código.

---

1. Papel e objetivo

Você é um engenheiro Android sênior, especialista em Kotlin, processamento de vídeo (Media3/MediaCodec), OCR on-device e arquitetura limpa. Quero que você projete e implemente **um aplicativo Android nativo completo** que:

1. Recebe um **vídeo já baixado** no aparelho.
2. Detecta as **legendas embutidas na imagem** do vídeo (hardcoded/burned-in, normalmente caixas de diálogo desenhadas no próprio vídeo, sem faixa de legenda separada).
3. Extrai o texto por **OCR**, agrupa por fala com tempo de início e fim, e **traduz** para o idioma escolhido.
4. Entrega o resultado de **duas formas** (o usuário escolhe):
   - **Modo A (principal): gerar arquivo de legenda** (.srt e .vtt) com o texto traduzido, e opcionalmente embutir como faixa de legenda (soft sub) em um novo .mkv/.mp4.
   - **Modo B (avançado): substituir o texto no vídeo** (burn-in): cobrir a legenda original e desenhar a tradução no lugar, gerando um novo vídeo.

Tudo deve funcionar **offline por padrão** (OCR e tradução on-device), com tradução online opcional e plugável.

2. Stack e requisitos técnicos

- Linguagem: **Kotlin** (sem Java, exceto onde uma lib exigir).
- UI: **Jetpack Compose** + Material 3, tema claro/escuro, suporte a portrait e landscape.
- Arquitetura: **MVVM + Clean Architecture** (camadas `ui`, `domain`, `data`), injeção de dependência com **Hilt**.
- Assíncrono: **Kotlin Coroutines + Flow**.
- Persistência: **Room** (projetos, falas, configurações) e **DataStore** (preferências).
- Trabalho em segundo plano: **WorkManager** com **foreground service** (tipo `dataSync` ou `mediaProcessing`) para processamentos longos, com notificação de progresso e botão de cancelar.

- Player e vídeo: **AndroidX Media3** (ExoPlayer para pré-visualização; Transformer para renderização/burn-in quando possível).
- Build: Gradle Kotlin DSL com **version catalog** (`libs.versions.toml`).
- minSdk **26**, targetSdk e compileSdk **mais recentes estáveis**.
- Arquiteturas: arm64-v8a e armeabi-v7a (x86_64 só para emulador).
- Sem dependências abandonadas. Se uma biblioteca estiver descontinuada (por exemplo o FFmpegKit original), use um **fork mantido** ou a alternativa nativa (Media3 Transformer) e explique a escolha.

3. Fluxo do usuário (telas)

1. **Início / Projetos**: lista de projetos anteriores com miniatura, status e data; botão "Novo projeto".
2. **Seleção do vídeo**: usar **Storage Access Framework** (`ActivityResultContracts.OpenDocument`) ou Photo Picker. Não pedir permissões amplas de armazenamento. Persistir a permissão de URI.
3. **Configuração do projeto**:
   - Idioma original (com opção **Detectar automaticamente**) e idioma de destino.
   - **Seleção da região da legenda** diretamente sobre um frame do vídeo (retângulo ajustável por arrastar/pinçar). Opção "Detectar região automaticamente" (analisar vários frames e achar a faixa onde o texto aparece com mais frequência, normalmente a parte inferior).
   - Suporte a **múltiplas regiões** (por exemplo, legenda inferior e caixas de diálogo em outro ponto da tela).
   - Taxa de amostragem de frames (padrão 3 fps, ajustável de 1 a 10).
   - Modo de saída: A (legenda), B (burn-in) ou ambos.
   - Qualidade/sensibilidade do OCR e motor de tradução.
4. **Processamento**: barra de progresso por etapa (extração, OCR, agrupamento, tradução, renderização), tempo estimado, botão pausar/cancelar, e funcionamento com a tela desligada.
5. **Revisão e edição** (essencial): linha do tempo com a lista de falas (texto original, tradução, início, fim). O usuário pode editar texto e tempos, mesclar ou dividir falas, apagar falsos positivos, reexecutar a tradução de uma fala e ver a **pré-visualização no player** com a legenda sobreposta em tempo real.
6. **Exportação**: salvar/compartilhar .srt, .vtt e o vídeo final; escolher pasta via SAF; abrir no player externo.
7. **Configurações**: motores de tradução, chaves de API (armazenadas com criptografia), glossário, estilo da legenda, cache, idioma do app, política de privacidade.

4. Pipeline de processamento (núcleo)

4.1 Extração de frames

- Usar `MediaMetadataRetriever.getFrameAtTime` com `OPTION_CLOSEST` para amostragem simples, ou `MediaExtractor` + `MediaCodec` com `ImageReader`/Surface para melhor desempenho em vídeos longos.
- Recortar **somente a região de interesse** logo após a decodificação para economizar memória.
- Reduzir o consumo: reutilizar bitmaps (pool), nunca manter frames inteiros em memória, processar em lotes pequenos.
- Respeitar a **rotação** do vídeo e vídeos com proporções incomuns, HDR e VFR (frame rate variável).

4.2 Pré-processamento da imagem

- Converter para tons de cinza, aumentar contraste, aplicar limiarização adaptativa (Otsu ou adaptativa) e, se necessário, inverter (texto claro em fundo escuro e vice-versa).
- Ampliar (upscale) regiões com texto pequeno antes do OCR.
- Tentar variantes (normal, invertida, binarizada) e **escolher a de maior confiança**.
- Suportar texto com contorno (stroke) e sombra, comuns em legendas e jogos.

4.3 Detecção de mudança (economia de OCR)

- Comparar a região do frame atual com a anterior (hash perceptual ou diferença média de pixels). **Só rodar OCR quando houver mudança relevante.**
- Reaproveitar o resultado quando o frame for praticamente igual.

4.4 OCR

- Motor padrão: **ML Kit Text Recognition v2** (on-device), com os modelos para latim, chinês, japonês, coreano e devanágari, carregados conforme o idioma escolhido.
- Interface `OcrEngine` abstrata para permitir um motor alternativo (por exemplo **Tesseract** via biblioteca mantida) quando o ML Kit não cobrir o idioma ou a fonte.
- Retornar texto, caixas delimitadoras (por linha/bloco) e confiança. Descartar resultados abaixo de um limite configurável.
- Normalizar o texto: remover quebras de linha indevidas, espaços duplicados, caracteres de lixo e corrigir confusões comuns (O/0, l/I) com base no idioma.

4.5 Agrupamento em falas (segmentação temporal)

- Uma fala começa quando um texto estável aparece e termina quando ele muda ou some.
- Considerar textos "iguais" quando a similaridade (Levenshtein normalizada) for acima de um limiar (ex.: 0,85), para tolerar erros de OCR entre frames.
- Para cada fala, escolher o **texto mais frequente/de maior confiança** entre os frames do grupo (votação).
- Definir início = primeiro frame com o texto; fim = último frame antes de mudar. Refinar os limites com uma busca binária entre amostras para precisão de frame.
- Ignorar falas muito curtas (ruído) abaixo de um mínimo configurável (ex.: 300 ms) e unir fragmentos adjacentes do mesmo texto.
- Tratar legendas de duas linhas e múltiplos falantes (cores diferentes, caixas diferentes) quando houver múltiplas regiões.

4.6 Tradução

- Interface `Translator` abstrata com implementações:
  1. **ML Kit Translate** (offline; baixar modelos de idioma sob demanda, mostrar progresso e permitir gerenciar/apagar modelos).
  2. **Tradutor online plugável** (DeepL, Google Cloud Translation ou um LLM via API), com chave do usuário e desativado por padrão.
- Traduzir **em lote** e **com contexto** (enviar falas vizinhas) para manter coerência de gênero, pronomes e nomes.
- **Glossário e lista de "não traduzir"** (nomes próprios, termos de jogo) editáveis pelo usuário.
- **Cache** de traduções em Room (chave: texto original + par de idiomas + motor).
- Preservar tags e formatação básica (itálico, quebras de linha) e respeitar um limite de caracteres por linha (ex.: 42) com quebra inteligente.
- Tratamento de erros e limites de taxa com **retry exponencial**.

4.7 Geração de legenda (Modo A)

- Gerar **SRT** e **WebVTT** corretos: numeração, formato de tempo `HH:MM:SS,mmm`, codificação **UTF-8 (com BOM opcional)**, quebras de linha estilo Windows opcionais.
- Opção de gerar também a legenda original e uma legenda **bilíngue** (duas linhas).
- Opção de **embutir como faixa de legenda** (soft sub) em contêiner MKV/MP4 sem re-encodar o vídeo (mux/remux), com o idioma marcado nos metadados.

- Opção de **cobrir a legenda original** na reprodução (overlay opaco) quando o usuário assistir dentro do app.

4.8 Substituição no vídeo (Modo B / burn-in)

- Para cada fala, durante seu intervalo de tempo:
  1. **Apagar o texto original**: preencher a caixa com a cor de fundo estimada (mediana das bordas da caixa) ou aplicar **inpainting simples**; oferecer a alternativa de uma tarja sólida/semitransparente.
  2. **Desenhar o texto traduzido** centralizado na região, com fonte, cor, contorno e tamanho que **se ajustem automaticamente** para caber na caixa (auto-fit com quebra de linha).
- Renderização preferencial com **Media3 Transformer + efeitos de overlay (OpenGL)**, mantendo o áudio e as demais faixas intactos. Alternativa: FFmpeg (fork mantido) com filtros `drawbox` e `drawtext` / `subtitles`.
- Manter resolução, frame rate e, quando possível, bitrate e codec do original; oferecer presets de qualidade (rápido/equilibrado/alta qualidade).
- Aceleração por **hardware encoder** (MediaCodec) com fallback para software.
- Avisar o usuário sobre o tempo e o espaço necessários antes de iniciar e checar o **espaço livre em disco**.

5. Desempenho, bateria e estabilidade

- Processar vídeos longos (2 horas ou mais) sem estourar a memória: pipeline em streaming, com limites de concorrência (`Dispatchers.Default` limitado, semáforo).
- **Checkpoint e retomada**: salvar o progresso no Room a cada N falas/segundos; se o app for morto ou cancelado, retomar de onde parou.
- Monitorar temperatura/bateria (`PowerManager` , `BatteryManager`) e reduzir a taxa de amostragem ou pausar se houver superaquecimento ou bateria crítica; opção "processar apenas carregando".
- Evitar vazamentos de bitmap e de recursos do MediaCodec; liberar tudo em `finally`.
- Suportar Android 8 a versões mais recentes, tratando mudanças de comportamento (restrições de serviços em primeiro plano, permissão de notificações no Android 13+, restrições de armazenamento).
- Testar em aparelhos de baixa memória (3 a 4 GB de RAM).

6. Privacidade, segurança e conformidade

- Processamento **local por padrão**; deixar explícito quando algum dado sair do aparelho (tradução online) e exigir consentimento.
- Chaves de API em **EncryptedSharedPreferences/Keystore**, nunca no código ou no log.
- Sem rastreamento invasivo; analytics opcionais e anonimizados.
- Pedir apenas permissões estritamente necessárias (notificações, SAF para arquivos).
- Tela de **aviso de direitos autorais**: o usuário deve ter o direito de modificar o vídeo.
- Tratar arquivos temporários com cuidado: pasta de cache do app e limpeza automática.

7. Tratamento de erros e casos de borda

Cobrir e mostrar mensagens claras para:

- Vídeo corrompido, codec não suportado, DRM, arquivo movido ou sem permissão.
- Nenhum texto detectado (sugerir ajustar a região ou o idioma).
- OCR de baixa confiança (marcar falas para revisão manual).
- Falha de rede, cota de API excedida, modelo de idioma não baixado.
- Pouco espaço em disco, bateria baixa, app em segundo plano, rotação de tela, mudança de configuração.
- Legendas que mudam de posição, fade in/out, animações e texto sobre fundos complexos.
- Idiomas RTL (árabe, hebraico), CJK e emojis na renderização do texto traduzido (fontes com fallback adequado).

8. Acessibilidade e internacionalização

- Interface do app traduzida (pelo menos português do Brasil e inglês) usando `strings.xml`.
- Compatível com TalkBack, tamanhos de fonte grandes e contraste adequado.
- Estilo da legenda configurável: fonte, tamanho, cor, contorno, sombra, fundo e posição.

9. Testes e qualidade

- **Testes unitários** (JUnit + MockK + Turbine) para: agrupamento de falas, similaridade de texto, geração de SRT/VTT, quebra de linha, cache de tradução.
- **Testes instrumentados** do fluxo principal e de UI (Compose Testing).

- Um conjunto de vídeos de teste curtos com legendas conhecidas para medir **taxa de acerto do OCR** e erro de temporização (meta: erro de tempo menor que 200 ms; precisão do texto acima de 95% em legendas limpas).
- Lint, ktlint/detekt configurados e **CI** (GitHub Actions) com build e testes.
- Logs estruturados (Timber) apenas em debug.

10. Entregáveis esperados da sua resposta

1. **Visão geral da arquitetura** com diagrama em texto e justificativa das escolhas.
2. **Estrutura completa de pastas/módulos** do projeto.
3. **Arquivos Gradle** completos (`libs.versions.toml` , `build.gradle.kts` do projeto e do app) com todas as dependências e versões.
4. `AndroidManifest.xml` com permissões, serviços e configurações necessárias.
5. **Código completo e compilável**, arquivo por arquivo, de: modelos e entidades Room, DAOs, repositórios, use cases, `OcrEngine` , `Translator` , extração de frames, detecção de mudança, agrupamento de falas, geradores SRT/VTT, Workers/serviço em primeiro plano, ViewModels e telas Compose (todas as da seção 3).
6. Implementação do **Modo A completo** e do **Modo B** (com a abordagem escolhida e suas limitações explicadas).
7. **Testes unitários** principais.
8. Instruções de **build, execução e teste** passo a passo, e uma lista de pontos de melhoria futura.

11. Regras de resposta

- Entregue em **etapas** para não truncar o código: primeiro arquitetura e Gradle; depois cada camada; ao final, integração. Pergunte "continuar?" ao fim de cada etapa se o limite de tamanho exigir.
- Código **completo, sem trechos omitidos** (nada de "..." ou "implemente aqui"), com comentários objetivos nas partes complexas.
- Se alguma decisão for ambígua, escolha a opção mais robusta, **declare a suposição** em uma linha e siga em frente.
- Aponte explicitamente riscos técnicos, limitações (ex.: qualidade do OCR em fontes estilizadas, tempo do burn-in em aparelhos fracos) e alternativas.
- Use as APIs atuais e **não invente classes ou métodos**; se não tiver certeza de uma API, diga e ofereça a alternativa verificada.

12. Fases de desenvolvimento sugeridas

1. **Fase 1 (MVP)**: seleção de vídeo, região manual, extração de frames, OCR, agrupamento, tradução offline, exportação SRT.
2. **Fase 2**: tela de revisão/edição com player, VTT, glossário, cache, retomada de processamento.
3. **Fase 3**: detecção automática de região e múltiplas regiões, tradutor online plugável, legenda soft sub embutida.
4. **Fase 4**: Modo B (burn-in) com Media3 Transformer, presets de qualidade e otimizações de bateria/temperatura.

Comece pela **Fase 1**, entregando o projeto completo e funcional dessa fase antes de avançar.